

Kubernetes (K8s)

O que é Kubernetes?

- **Kubernetes** ([/ˌk\(j\)uːbərˈnɛtɪs, -ˈnɛɪtɪs, -ˈnɛɪtiːz, -ˈnɛtiːz/](#)), also known as **K8s**, is an [open-source container orchestration](#) system for automating [software deployment](#), scaling, and management.^{[3][4]} Originally designed by [Google](#), the project is now maintained by a worldwide community of contributors, and the trademark is held by the [Cloud Native Computing Foundation](#).
- The name *Kubernetes* comes from the [Ancient Greek](#) term κυβερνήτης, *kubernētēs* (helmsman, pilot), which is also the origin of the words [cybernetics](#) and (through Latin) [governor](#). "Kubernetes" is often abbreviated with the [numerical contraction](#) "K8s", meaning "the letter K, followed by 8 letters, followed by s".^[5]

História

- Kubernetes was announced by Google on June 6, 2014.^[9] The project was conceived and created by Google employees Joe Beda, Brendan Burns, and Craig McLuckie. Others at Google soon joined to help build the project including Ville Aikas, Dawn Chen, Brian Grant, Tim Hockin, and Daniel Smith.^{[10][11]} Other companies such as [Red Hat](#) and [CoreOS](#) joined the effort soon after, with notable contributors such as Clayton Coleman and [Kelsey Hightower](#).^[9]

Por que Kubernetes?

- Containers resolvem empacotamento e portabilidade.
- Em aplicações distribuídas surgem novos problemas:
 - Como executar muitos containers?
 - Como reiniciar containers que falharam?
 - Como distribuir tráfego?
 - Como escalar?
 - Como atualizar sem interromper o serviço?

O que é Kubernetes?

- Plataforma open source para automatizar implantação, gerenciamento e escalonamento de aplicações containerizadas.
- Abstrai a infraestrutura e permite declarar o estado desejado da aplicação.
- Também é conhecido como K8s.

Containers × Kubernetes

- Container: unidade de execução de uma aplicação.
- Docker: ferramenta/ecossistema para criar e executar containers.
- Kubernetes: coordena containers distribuídos em um cluster.
- Kubernetes não substitui o container; ele o orquestra.

Cluster Kubernetes

- Um **cluster** é formado por máquinas chamadas **nodes**.
- **Control Plane**: toma decisões e mantém o estado do cluster.
- **Worker Nodes**: executam as aplicações.
- Os componentes comunicam-se por meio da **API** do Kubernetes.

Arquitetura básica

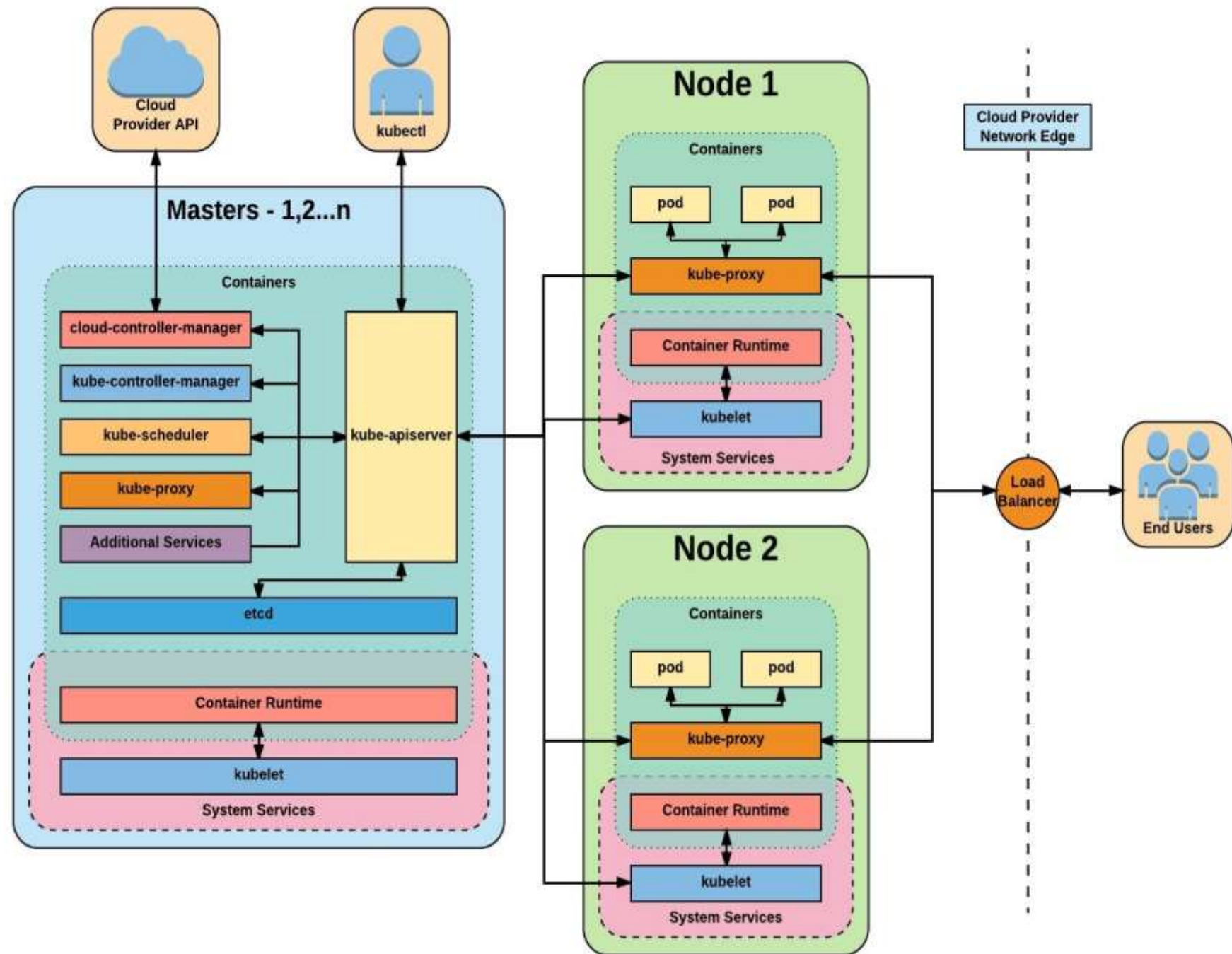
■ Control Plane

- kube-apiserver: API central
- etcd: armazenamento do estado do cluster
- scheduler: escolhe onde executar Pods
- controller manager: reconcilia o estado desejado

■ Worker

- Kubelet: gerencia Pods no node
- container runtime: executa containers
- kube-proxy: componentes de rede

Arquitetura Básica



Control Plane

- Gerencia estado do cluster (API Server, Scheduler, Controller Manager)
- **API Server:** Interface para comunicação com **cluster**
- **Scheduler:** Aloca **pods** em **nodes** disponíveis
- **Controller Manager:** Executa controladores que mantêm estado desejado

Node

- **Kubelet:** Agente que garante containers rodem em pods
- **Container Runtime:** Docker, containerd ou CRI-O
- **kube-proxy:** Gerencia networking entre pods

Pod

- Menor unidade de execução do Kubernetes.
- Normalmente contém um container.
- Containers do mesmo Pod compartilham rede e podem compartilhar volumes.
- Pods são efêmeros; não devemos depender de um Pod específico permanecer ativo.

Deployment

- Define como uma aplicação deve ser executada.
- Permite especificar imagem, número de réplicas, recursos, variáveis de ambiente e estratégia de atualização.
- O **Deployment** cria e gerencia **ReplicaSets**, que mantêm os **Pods** desejados.

Service

- **Pods** podem ser criados e destruídos dinamicamente.
- Seus endereços IP não devem ser usados como identidade permanente.
- **Service** fornece um ponto de acesso estável para um conjunto de **Pods**.
- Tipos comuns: **ClusterIP**, **NodePort** e **LoadBalancer**.

Estado desejado × estado atual

- Kubernetes trabalha com o princípio de reconciliação.
- O administrador declara o estado desejado.
- Controladores observam o estado atual.
- Se houver diferença, Kubernetes tenta corrigi-la.
- Exemplo: desejamos 3 réplicas; se um Pod morrer, outro será criado.

Escalonamento

- Escalonamento horizontal: aumentar ou reduzir o número de Pods.
 - Exemplo: de 3 até 10 réplicas.
- Pode ser manual ou automático.
- Horizontal Pod Autoscaler (HPA) pode usar CPU, memória e outras métricas disponíveis.

Self-healing

- Kubernetes monitora os recursos declarados.
- Se um container falhar, o runtime pode reiniciá-lo.
- Se um Pod desaparecer, um controlador pode criar outro.
- Workloads podem ser reagendados conforme as condições e configurações do cluster.

Atualizações e Rollback

- Deployments permitem atualizar versões de uma aplicação.
- Rolling Update substitui Pods gradualmente.
- Isso reduz indisponibilidade durante uma atualização.
- Se uma nova versão apresentar problemas, é possível realizar rollback.

ConfigMap e Secret

- **ConfigMap**: configurações que não são consideradas secretas.
- **Secret**: dados sensíveis, como **credenciais** e **tokens**.
- Podem ser usados por variáveis de ambiente ou volumes.
- **Importante**: Secret não significa automaticamente criptografado.

Volumes e persistência

- **Containers** são, em geral, **efêmeros**.
- Dados que precisam sobreviver à recriação de **Pods** devem usar armazenamento persistente.
- Kubernetes fornece **PersistentVolume** (PV) e **PersistentVolumeClaim** (PVC).

Namespaces

- **Namespaces** permitem separar recursos dentro de um cluster.
- Podem ser usados para organização, isolamento lógico e controle de acesso.
- Exemplos: desenvolvimento, testes, produção e diferentes equipes.

Labels e Selectors

- **Labels** são pares chave/valor associados a objetos.
 - Exemplo: `app=web`
- **Selectors** permitem selecionar objetos com determinados labels.
- **Services e Deployments** usam **selectors** para relacionar recursos.

YAML e objetos Kubernetes

- Recursos Kubernetes podem ser descritos declarativamente em YAML.

- Exemplo conceitual:

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: web
```

```
spec:
```

```
  replicas: 3
```

- O YAML descreve o estado desejado.

kubectl

- **kubectl** é a principal ferramenta de linha de comando para interagir com a **API** do Kubernetes.

- Comandos fundamentais:

```
kubectl get pods
```

```
kubectl get deployments
```

```
kubectl get services
```

```
kubectl describe pod <nome>
```

```
kubectl logs <nome>
```

```
kubectl apply -f arquivo.yaml
```

Fluxo de uma aplicação

1. Criar uma imagem de container.
2. Declarar um Deployment.
3. Kubernetes cria os Pods.
4. Um Service fornece acesso estável.
5. O scheduler distribui Pods pelos nodes.
6. Controllers mantêm o estado desejado.
7. HPA pode ajustar a capacidade.

Kubernetes não é...

- Um sistema operacional.
- Uma máquina virtual.
- Um servidor web.
- Um banco de dados.
- Uma ferramenta exclusiva para Docker.
- É uma plataforma de orquestração que coordena workloads containerizados.

Conceitos importantes

- Node: uma máquina (física ou virtual).
- Cluster: conjunto de nodes.
- Pod: unidade básica de execução.
- Deployment: gerencia réplicas e atualizações.
- Service: acesso estável a Pods.
- ConfigMap/Secret: configuração e dados sensíveis.
- Namespace: separação lógica.
- kubectl: interface de administração.

Próximos passos

- Instalar um cluster local.
- Criar Pods e Deployments.
- Expor aplicações com Services.
- Trabalhar com ConfigMaps, Secrets e volumes.
- Explorar namespaces e recursos.
- Configurar probes e recursos de CPU/memória.
- Experimentar escalonamento e rolling updates.
- Depois: networking, observabilidade e alta disponibilidade.

Kubernetes (K8s)